JanDrishti (■■ ■■■■■■)

AI-Based Predictive Urban Infrastructure Intelligence Platform

Document Classification: Complete Technical Architecture, System Engineering & Operational Guide
Author / Lead: Senior Software Architect & Collaborative Technical Lead
Team Scope: 5 Concurrent Engineers (Frontend, AI/ML, Backend, Database/GIS, Telemetry/QA)
Version: 1.0.0-hackathon (Monorepo Production Baseline)
Generated Date: October 2026
Core Repositories: Modular Decoupled Monorepo (React, FastAPI, YOLOv8, MongoDB Atlas, Leaflet)

EXECUTIVE SUMMARY & PROBLEM STATEMENT:
Modern metropolitan roads deteriorate rapidly due to heavy commercial freight transit and torrential monsoon runoff. Traditional municipal road inspections suffer from three fatal bottlenecks:

1. Slow & Subjective Manual Surveys: Municipalities rely on physical complaint helplines and sporadic manual audits, discovering potholes only after severe traffic accidents or public outcry.
2. Inability to Predict Failure: Road surface cracks (longitudinal, transverse, alligator) are treated in isolation without fusing vehicular vibration shocks and environmental hydrology factors.
3. Unverified Contractor Work (Ghost Repairs): Tens of millions of municipal funds are disbursed without scientific verification that contractor resurfacing was successfully completed.

The JanDrishti Solution: An autonomous, closed-loop urban intelligence platform that converts municipal vehicle fleets (buses, sanitation trucks, patrol vans) into edge-sensing probes. Camera video frames and 3-axis accelerometer telemetry (Z-shock) are streamed into a YOLOv8 computer vision detector, fused with geospatial environmental factors to calculate an **Infrastructure Health Score (IHS)**, rendered on a live municipal Leaflet GIS map, and audited via an AI before/after repair verification engine.

1. End-to-End Functional Architecture

The JanDrishti system operates on a synchronized 9-stage closed-loop data pipeline. Every component communicates via strictly typed API contracts, allowing all modules to operate independently while preserving full-stack data integrity.

Stage 1: Vehicle Data Collection (Dev 5)	Public transit buses, waste collection trucks, and municipal patrol bikes are equipped with smartphone/dashcam units capturing HD road video, GPS coordinates, and 3-axis accelerometer telemetry (Z-shock vibration spikes).
Stage 2: AI Road Damage Detection (Dev 2)	Video frames are processed through YOLOv8 models trained on the Global Road Damage Dataset (RDD), identifying Potholes (D00), Alligator Cracking (D10), Longitudinal Cracks (D20), and Transverse Cracks (D40). OpenCV calculates pixel surface area in square meters.
Stage 3: GPS Snapping & Spatial Routing (Dev 4)	Raw GPS telemetry coordinates are snapped to the nearest managed road stretch using geospatial MongoDB 2dsphere indexing and Euclidean thresholding.
Stage 4: MongoDB Atlas Persistence (Dev 4)	Cleaned observation records containing detected bounding boxes, severity classifications, and telemetry shocks are persisted into the observations collection.
Stage 5: Multi-Hazard Fusion (Dev 2 & 3)	The system executes a multi-variate fusion algorithm combining surface damage area, vertical accelerometer impact shocks (Z-shock > 3.0 m/s²), monsoon water runoff vulnerability, and heavy vehicle transit frequency.
Stage 6: Infrastructure Health Score (IHS) (Dev 2 & 3)	Computes a normalized health score (0–100 scale, where 100 is pristine and 0 is complete structural failure) and calculates 30-day failure probability.
Stage 7: Risk Classification & Remediation Cost (Dev 2 & 3)	Categorizes the road stretch into actionable municipal tiers: Routine Monitoring, Patching Required, or Emergency Resurfacing, with automated algorithmic remediation cost estimations in INR.
Stage 8: Municipal Command Dashboard (Dev 1)	Interactive Leaflet / OpenStreetMap GIS displays the complete urban road network with color-coded health statuses (Green >75, Amber 50-75, Red <50), clickable popups, and citywide KPI metric cards.
Stage 9: Closed-Loop Repair Verification (Dev 5 & 1)	Municipal work orders are issued. Upon contractor repair, a post-resurfacing photo is uploaded; the AI verification engine runs surface difference checks to confirm hazard elimination before authorizing status closure (verified_closed).

2. Deep-Dive: Technologies Used & Technical Stack

JanDrishti is designed as a modular monorepo. Each technological choice was selected specifically for high throughput, asynchronous I/O, low-latency rendering, and rapid hackathon delivery:

Layer	Technology	Role & Engineering Justification
Frontend UI	`React 18 + Vite 5`	High-performance Single Page Application (SPA) with instant Hot Module Replacement (HMR), decoupled service layers, and reactive state management.
Styling	`Tailwind CSS v3`	Utility-first design system enabling rapid responsive UI, dark-mode styling (Slate-900 palette), and consistent municipal portal aesthetics.
GIS Mapping	`Leaflet JS + OpenStreetMap`	Client-side interactive vector and raster GIS engine. Renders GeoJSON LineStrings with dynamic color coding, popup bindings, and hazard circle markers without expensive Google Maps API licensing.
Backend Framework	`Python 3 + FastAPI`	High-throughput asynchronous REST framework powered by Starlette and Uvicorn. Native async/await handles concurrent vehicle telemetry streams with sub-millisecond overhead.
Validation & Serialization	`Pydantic V2`	Strict runtime request/response validation, automated OpenAPI/Swagger generation, and schema enforcement matching the team's shared API Contract.
Database Engine	`MongoDB Atlas`	Cloud-native NoSQL document database providing dynamic schema agility for semi-structured sensor telemetry and high-speed write performance.
Geospatial Engine	`GeoJSON + 2dsphere Indexes`	Native spherical geospatial calculations enabling spatial querying ($near, $geoIntersects) to match vehicle GPS points to road geometries.
Computer Vision AI	`Ultralytics YOLOv8`	State-of-the-art single-stage convolutional object detection network capable of real-time 30+ FPS edge inference on road surface damage.
Image Processing	`OpenCV (cv2)`	CLAHE (Contrast Limited Adaptive Histogram Equalization) preprocessing to normalize intense tropical sunlight and deep tree shadows, plus bounding box annotation.
Predictive AI	`Multi-Hazard Fusion Algorithm`	Multi-factor mathematical scoring model fusing visual damage area, accelerometer Z-shock spikes, and monsoon drainage vulnerability indices.
Testing Suite	`Pytest + TestClient`	Automated test harnesses validating REST endpoints, AI detection fallback logic, and 9-stage end-to-end integration lifecycle flows.

3. Mathematical Modeling: Multi-Hazard Fusion Engine

A central architectural innovation of JanDrishti is moving beyond superficial computer vision counts. A pothole in a dry, low-traffic alley presents far less systemic danger than a pothole on an arterial corridor subject to monsoon flooding and heavy commercial axles.

The **Multi-Hazard Fusion Engine** synthesizes four normalized environmental vectors into a unified **Composite Risk Index ($R_{ ext{composite}}$)**:

1. Normalized Factor Formulations:
- **Pothole Severity Impact (I_p):** min(1.0, (Count × 0.3) + (Total Area m² × 0.4))
- **Crack Fatigue Impact (I_c):** min(1.0, Crack Count × 0.25)
- **Vehicular Shock Vibration (I_v):** min(1.0, max(0.0, ($Z_{ ext{shock}} - 1.0) / 4.0))
- **Monsoon Runoff Vulnerability (I_m):** Hydrological index (0.0: Elevated ridge, 1.0: Flood-prone basin)

2. Composite Multi-Hazard Risk Equation:

$$R_{composite} = [(w_p \cdot I_p) + (w_c \cdot I_c) + (w_v \cdot I_v) + (w_m \cdot I_m)] \times (0.8 + 0.2 \cdot F_{traffic})$$

Default Weights: $w_p = 0.40$, $w_c = 0.25$, $w_v = 0.20$, $w_m = 0.15$

3. Infrastructure Health Score (IHS):

$$IHS = \max(5.0, 100.0 - (R_{composite} \times 90.0))$$

- **IHS ≥ 75:** Condition Good (Green) → Routine Monitoring
- **50 ≤ IHS < 75:** Condition Fair (Amber) → Patching Required
- **IHS < 50:** Condition Critical (Red) → Emergency Resurfacing

4. Five-Developer Team Responsibilities & Isolation

To prevent code collisions and integration gridlocks during fast-paced hackathons, the monorepo establishes rigid directory boundaries and contract-first interfaces:

Developer	Branch	Module Directory	Core Deliverables & Scope
Dev 1: Frontend	`feature/frontend`	`frontend/`	React SPA, Leaflet GIS integration, Metric cards, Risk analytics panels, Repair audit view, and mock fallback handlers.
Dev 2: AI / ML	`feature/ai-detection`	`ai-engine/`	YOLOv8 damage classification (D00-D40), OpenCV CLAHE image preprocessor, Multi-Hazard Risk Fusion, and offline CLI runner.
Dev 3: Backend	`feature/backend`	`backend/app/routers/`	FastAPI REST routing, JWT authentication, multipart ingestion endpoints, API contract enforcement, and global error envelope.
Dev 4: Database/GIS	`feature/database-gis`	`backend/app/database/`	MongoDB Atlas connection manager, Pydantic collection schemas, 2dsphere indexes, GeoJSON road endpoints, and database seeder.
Dev 5: Data & QA	`feature/data-collection`	`tests/ & scripts/`	Vehicle GPS & Z-shock telemetry simulator, Pytest test suites, contractor repair verification endpoint, and integration testing.

5. Shared Database Collections & API Specifications

5.1 MongoDB Atlas Core Collections (Developer 4):

- `users`: User accounts with role-based access: admin, municipal_officer, field_worker, contractor.
- `vehicles`: Fleet asset registry with vehicle_type (bus, waste_truck, van) and GeoJSON Point coordinates.
- `observations`: Raw telemetry and camera frames containing detected bounding boxes, severity, and telemetry_z_shock.
- `road_segments`: Managed urban stretches with GeoJSON LineString coordinates, health_score (0-100), and hazard counts.
- `risk_assessments`: Multi-Hazard predictive calculations, 30-day failure probabilities, and estimated repair costs in INR.
- `repair_records`: Closed-loop verification tickets with before/after photos and AI surface verification scores.

5.2 Primary REST API Endpoints (docs/API_CONTRACT.md):

HTTP	Endpoint Route	Payload / Parameters	Description & Response
GET	**/api/v1/health**	**None**	Returns status of backend, MongoDB Atlas, and AI engine.
POST	**/api/v1/auth/login**	**{email, password}**	Authenticates user and returns JWT Bearer token.
POST	**/api/v1/observations/upload**	**Multipart (image, GPS, Z-shock)**	Ingests road frame, runs YOLO detection, stores observation.
GET	**/api/v1/roads/geojson**	**?ward_id (optional)**	Returns GeoJSON FeatureCollection with health color coding.
GET	**/api/v1/analytics/overview**	**None**	Citywide KPI metrics, hazard breakdown, and ward rankings.
POST	**/api/v1/risk/evaluate/{id}**	**{segment_id}**	Executes Multi-Hazard Fusion algorithm on target stretch.
POST	**/api/v1/repairs/verify/{id}**	**Multipart (after_file)**	AI compares before/after photos to verify pothole elimination.

6. Operational Commands & Verification Guide

Running Frontend: `cd frontend && npm run dev` → http://localhost:5199
Running Backend: `.venv/bin/uvicorn app.main:app --app-dir backend --port 8199` → http://localhost:8199/docs
Simulating Fleet Ingestion: `python3 scripts/simulate_vehicle_telemetry.py 5`
Running AI CLI Engine: `python3 ai-engine/cli.py --image road.jpg --z-shock 3.4`
Automated Test Suite: `.venv/bin/pytest tests/` (All 3 test suites: Backend, AI, Integration pass 100%)